Week 4, Session 1 — What Is an AI Agent?

IDSC 4101: Agentic AI for Business Applications

Session type: Lecture + Diagram Walk-Through + Agent Blueprint Activity **Tools introduced:** None new (conceptual session — builds foundation for Week 5 tool work)

SLIDE DECK

SLIDE 1 — Title Slide

Title: What Is an AI Agent? **Subtitle:** From AI That Responds to AI That Acts **Course:** IDSC 4101: Agentic AI for Business Applications | Week 4, Session 1

Speaker Notes: We've spent three weeks understanding generative AI — what it is, how to use it, how businesses are deploying it for workflows and functions. You know how to prompt well. You know what RAG is. You've seen AI automate marketing copy, financial summaries, and HR screening. Today we cross a threshold. We're moving from AI that *responds* to AI that *acts*. An AI agent isn't just answering your question — it's deciding what to do next, using tools to do it, checking its own work, and looping until the goal is met. That shift from response to action is one of the most significant changes in how AI is being deployed right now. By the end of today, you'll be able to explain what an agent is, draw one, and evaluate whether one makes sense for a business problem. Let's go.

SLIDE 2 — Where We've Been, Where We're Going

Title: The Arc So Far

Bullets: - Weeks 1–2: Generative AI foundations — what it is, how it works, personal and team productivity - Week 3: AI for business functions — automating workflows, cross-functional deployment - **Week 4 (today): Agentic AI** — what agents are, how they reason, how to design them - Weeks 5–6: Building with agents — real systems, real failures, real business deployment - Week 7: Strategy, ethics, and where this goes next

Speaker Notes: We've been building a ramp. The first half of the course was about understanding generative AI well enough to use it confidently. Now we shift into the part of the course that's genuinely at the frontier — agentic AI. This is the technology that's getting the most investment right now, causing the most organizational disruption, and creating the most new business opportunities. It's also the most misunderstood. Today is the conceptual foundation. Don't worry about tools yet — we'll get hands-on in Week 5. Today is about building the mental model.

SLIDE 3 — The Core Distinction

Title: Generative AI vs. Agentic AI — One Key Difference

Two-column layout:

Generative AI	Agentic AI
You ask → it responds	It receives a goal → it figures out how to achieve it
One turn: prompt in, output out	Multiple turns: plan, act, check, adjust, repeat
Passive	Active
You decide what to do with the output	It decides what to do next
Example: “Draft a summary of this report”	Example: “Research our top 5 competitors and produce a comparison table”

Speaker Notes: This one distinction is worth spending time on, because it changes everything about how you evaluate and deploy these systems. A generative AI tool — a chatbot, a drafting assistant, a summarizer — waits for you. You tell it exactly what to do, it does it, and it stops. An agent has a goal and autonomy over how to achieve it. You say “find me the top 5 competitors in our market, summarize their pricing pages, and flag any that have launched a new product in the last 90 days.” The agent doesn’t wait for you to tell it each step. It figures out the steps, uses tools to execute them — web search, document reader, comparison formatter — and produces a result. That difference in autonomy is what makes agents powerful. It’s also what makes them risky.

SLIDE 4 — Agent Anatomy (Part 1): The Four Components

Title: What an Agent Is Made Of

Diagram description (draw on board or show as labeled diagram): Four boxes connected by arrows in a loop: 1. **Goal** → What the agent is trying to achieve (set by the user or another system) 2. **Tools** → What the agent can do: search the web, read files, send emails, call APIs, write to databases 3. **Memory** → What the agent remembers: the conversation so far, results of previous steps, facts it has retrieved 4. **Reasoning Loop** → The agent’s decision engine: “Given my goal and what I know so far, what should I do next?”

Speaker Notes: Let’s walk through this carefully, because this diagram is the foundation for everything else in the course. Start with the goal — this is set by a person, by a business process, or by another agent. It might be specific (“find all open invoices over 30 days”) or broad (“help onboard a new employee”). The agent doesn’t just receive the goal and respond once — it enters a loop. It reasons: what do I know? What do I need to find out? What tool should I use? It acts — runs a search, reads a document, sends a message. It checks the result. It updates its memory with what it learned. It reasons again. This loop continues until the goal is met or until the agent decides it can’t proceed. The human may not be in that loop at all — or they may be a checkpoint along the way. We’ll come back to that distinction in Session 2.

SLIDE 5 — Agent Anatomy (Part 2): The Reasoning Loop Up Close

Title: How an Agent Thinks — The ReAct Pattern

Step-by-step flow (show as numbered sequence): 1. **Observe:** What do I know right now? What has happened so far? 2. **Think:** What should I do next to get closer to the goal? 3. **Act:** Use a tool — search, read, write, call, calculate 4. **Observe again:** What did I get back? Does it move me toward the goal? 5. **Think again:** Is the goal met? Do I need to adjust? Do I need to hand off? 6. **Loop or terminate:** Continue if needed; stop and report when done

Speaker Notes: This is called the ReAct pattern — short for Reason and Act. It’s the dominant design pattern for agents right now, used in most of the commercial agent products you’ll encounter. What’s important for you as a business professional isn’t the technical name — it’s the logic. Agents aren’t running a single query.

They’re running a reasoning loop. That means their behavior is harder to predict than a simple AI response. Each step produces an output that changes what happens next. A wrong turn at step 3 can compound into a very wrong outcome by step 6. This is why the failure mode analysis we do in Session 2 is so important. For now, just internalize the loop: observe, think, act, repeat.

SLIDE 6 — Four Agentic Design Patterns

Title: How Agents Are Built to Work — Four Patterns

Bullets (with one-line description each): - **Reflection:** The agent reviews its own output before handing it off — asks “is this good enough?” and revises if not - **Tool Use:** The agent calls external systems — search engines, calculators, databases, APIs — to get information or take action it couldn’t do on its own - **Planning:** The agent breaks a complex goal into a sequence of sub-tasks, executes them in order, and adjusts the plan if something fails - **Handoff:** The agent routes a task to a human or to another agent when it hits a boundary — a decision it’s not authorized to make, a task requiring judgment it doesn’t have

Speaker Notes: These four patterns are the vocabulary you need to evaluate any agent system a vendor shows you or a team proposes building. Ask: does this agent reflect — and how do you know when the reflection is good enough? What tools does it have access to — and what happens if a tool fails? Does it plan or just react? And critically: when does it hand off, and to whom? The handoff pattern is the one most often missing from early deployments, and it’s frequently the source of failures. We’ll dig into that in Session 2.

SLIDE 7 — Types of Agents in Business

Title: The Agents You’ll Actually Encounter

Bullets: - **Task Agents:** Narrow scope, single function. Example: a research agent that pulls competitor data on demand. Reliable, low-risk, easy to evaluate. - **Conversational Agents:** Customer-facing AI that handles questions, routes issues, and escalates when needed. Example: a customer support agent for a SaaS product. - **Process Automation Agents:** Embedded in business workflows. Example: an agent that monitors accounts receivable, flags overdue invoices, and drafts follow-up emails. - **Multi-Agent Systems:** One orchestrator agent breaks a goal into sub-tasks and routes each to a specialized sub-agent. Example: a market research pipeline with separate agents for search, synthesis, and formatting.

Speaker Notes: These aren’t mutually exclusive — a real deployment might combine several types. But the taxonomy helps you ask the right questions. A task agent is probably low-stakes to start. A conversational agent is customer-facing, which raises the stakes significantly — what happens when it says something wrong or offensive? A process automation agent is embedded in operations, which means errors can cascade into real business impact before a human notices. And a multi-agent system amplifies all of these dynamics: more capability, more complexity, more ways to fail. When you evaluate a proposed agent deployment, start by asking: which type is this? That tells you a lot about the risk profile before you look at anything else. —

SLIDE 8 — Real Business Examples (Walk-Through)

Title: Three Agents in the Wild — What They Actually Do

Example 1: AI Sales Development Representative (SDR)

What it does: - Receives a list of target accounts from a CRM - Searches the web for recent news about each company (funding, leadership changes, product launches) - Reads the company's LinkedIn page and website - Drafts a personalized outreach email for each account - Flags accounts where it found no recent news (lower confidence) for human review - Sends approved emails, logs activity in CRM

What tools it uses: Web search, document reader, CRM API, email API

Where humans are involved: Review queue for flagged accounts; final send approval (depending on deployment)

Speaker Notes: This is a real deployment pattern — companies like Outreach, Apollo, and several startups are selling exactly this. What I want you to notice is the loop: the agent doesn't just draft one email. It sequences through the account list, runs a search for each, reads the results, decides what's relevant, drafts based on what it found, and self-evaluates (the flagging step). That flagging is a primitive form of reflection — the agent is asking itself "am I confident enough to proceed?" That's a design choice the developers made. In a less careful deployment, it would just send everything, including the poorly-researched ones. The presence or absence of that reflection step has real business consequences.

Example 2: Operations Exception Agent

What it does: - Monitors an inventory management system continuously - Flags any SKU that drops below a reorder threshold - Cross-references against sales velocity data to distinguish "normal low" from "urgent reorder" - Drafts a purchase order for urgent cases - Routes to a procurement manager for approval above a dollar threshold - Logs all actions and decisions to an audit trail

What tools it uses: Inventory database, sales database, PO generation system, email/Slack API

Where humans are involved: Approval for high-value orders; escalation for anomalies the agent flags as unusual

Speaker Notes: Notice the threshold logic — this agent hands off to a human based on a rule, not just when it "gets confused." That's intentional design. The developers decided that above a certain dollar value, a human must approve. That's a guardrail embedded into the agent's design. It's not AI safety in the abstract — it's a business decision about where human judgment needs to be in the loop. When you evaluate or design agent systems, one of your most important jobs is asking: what are the thresholds? What triggers a handoff? That conversation is not a technical one — it's a business one.

Example 3: Employee Onboarding Agent

What it does: - Answers new employee questions about policies, benefits, and systems (using company knowledge base) - Detects when a question falls outside its knowledge base and escalates to HR - Proactively sends reminders for onboarding tasks (complete I-9, set up direct deposit, enroll in benefits) - Logs questions it couldn't answer for HR to review and use to improve the knowledge base

What tools it uses: Knowledge base search, HR system API, email/Slack API

Where humans are involved: Escalation for edge cases; ongoing knowledge base maintenance

Speaker Notes: This one is interesting because the agent's failure mode isn't catastrophic — if it says something wrong about a benefit, the employee goes to HR, HR corrects it. The stakes are lower than the operations agent. That matters when you're deciding how much autonomy to give a system. The onboarding agent can be given more latitude because the error correction path is easy. The operations agent touches money, so the approval threshold is more conservative. Risk-calibrated autonomy — that's the principle. Not "how capable is the agent" but "what's the cost of it being wrong?"

SLIDE 9 — The Analogy Frame: Agent as New Employee

Title: Think of an Agent Like a Very Fast, Very Literal New Hire

Bullets: - A new employee follows instructions — but only as literally as they understand them - They'll use the tools they've been given — but not ones they haven't been shown - They'll make decisions within their authority — and escalate what's outside it - They make mistakes when the goal is ambiguous, when context is missing, or when the situation is novel - They get better over time — but only if someone reviews their work and gives feedback

The key question for any agent deployment: *If this were a new hire, would you give them this level of authority on day one?*

Speaker Notes: I find this analogy useful because it grounds the conversation in human intuition that business students already have. You wouldn't hand a new hire the company credit card and tell them to "handle vendor payments" without any oversight. But you might trust them to draft emails or summarize documents with minimal review. The same logic applies to agents. The question isn't "can this agent do it" — it can, technically. The question is "should we let it do it without oversight, given what we know about how it can fail?" That's a management question, not an engineering question. And it's exactly the question you'll be the ones asking in your careers.

SLIDE 10 — Course Connection: Agent Blueprint Activity Preview

Title: Your Turn — Design an Agent

What you'll do: - In teams of 3–4, pick a business problem you find genuinely interesting - Sketch an agent that could address it using today's framework - Define: Goal | Tools | Memory | Reasoning loop | Handoff points - Identify: What could go wrong? Where does a human need to be? - Share with the class — instructor gives live feedback on each

Why this matters: - This blueprint is the foundation for your Session 2 Failure Mode Analysis - It's also a preview of capstone-level thinking: evaluating AI deployment feasibility for a real business problem

Speaker Notes: We've covered a lot of conceptual ground. Now you're going to use it. The Agent Blueprint activity is deliberately open-ended — I'm not giving you a template beyond the four components we just covered. That's intentional. Part of what we're practicing is translating a messy business problem into a structured agent design. That translation skill is what distinguishes someone who can think about AI deployment from someone who just knows the vocabulary. I'll circulate and ask questions. When we debrief, I'll give feedback on each blueprint — I'm particularly interested in the handoff points, because that's where most early designs are weakest. Ready? Let's go.

SLIDE 11 — Activity Card (displayed during activity)

Title: Agent Blueprint — Team Activity

Prompt displayed on screen:

Your team has 20 minutes. Choose a business problem — any function, any industry. Design an agent to address it. Use this structure:

Component	Your Answer
Goal	What is the agent trying to accomplish? (one sentence)

Component	Your Answer
Tools	What does it need access to? (list 3–5)
Memory	What does it need to remember across steps?
Reasoning loop	What does one “cycle” look like? (describe the observe-think-act sequence)
Handoff	When does a human need to step in? What triggers the handoff?
Top risk	What’s the most likely way this agent fails or causes harm?

Be ready to share your blueprint with the class. One spokesperson per team, 2 minutes max.

Speaker Notes: Keep this slide up the entire time. Circulate, don’t stand at the front. The goal is to help teams who are stuck on the reasoning loop — most will define goal and tools easily, but struggle to describe the loop. Prompt them: “Walk me through what the agent does from the moment it receives the goal. Step by step. What does it do first? What does it look at? What decision does it make?” That walk-through usually unlocks the loop. For teams that have a very complex multi-step agent, nudge them to simplify: “What’s the core loop? What’s the minimum version that works?” Simplicity in the blueprint stage is a virtue.

SLIDE 12 — Debrief and Close

Title: What We Built Today

Bullets: - Generative AI responds. Agentic AI acts — plans, uses tools, loops, hands off. - Every agent has four components: goal, tools, memory, reasoning loop. - Four design patterns: reflection, tool use, planning, handoff. - Four types in business: task agents, conversational agents, process automation agents, multi-agent systems. - The key business question: *At what level of autonomy should this agent operate, and why?*

Coming up in Session 2: - Why agents fail — and how to design against it - Guardrails: what they are and how to specify them without code - Human-in-the-loop spectrum: from fully autonomous to human-approved - Failure Mode Analysis: your blueprint, under stress

Speaker Notes: Today was about building the mental model. I want you to leave with one idea above all others: agents are not magic, and they are not just “better chatbots.” They are systems that take actions in the world, and that makes them fundamentally different from everything we’ve studied so far. The same qualities that make them powerful — autonomy, persistence, tool access — are the qualities that make them dangerous when they’re wrong. Session 2 is where we stress-test that. Bring your blueprint. We’re going to break it. See you next time.

ACTIVITY PLAN

Activity: “Agent Blueprint” (20 minutes + 10-minute debrief)

Purpose: Translate the conceptual framework introduced in lecture into a structured agent design for a real business problem. Students practice the observe-think-act loop in design terms, and surface their own intuitions about where human oversight belongs — setting up the Failure Mode Analysis in Session 2.

Team configuration: - Teams of 3–4 students (use standing course teams if established, or quick random assignment) - One spokesperson per team for debrief

What students do:

Phase	Duration	Task
Team formation and problem selection	3 min	Pick a business problem — any function, any industry. Instructor prompt if stuck: "Think about a job at a company you'd want to work at. What's the most repetitive, information-heavy task someone does in that job?"
Blueprint sketching	17 min	Complete the six-field blueprint on the activity card (Slide 11). Use paper, whiteboard, or a shared doc — whatever is fastest.
Team debrief: share-out	10 min	Each team's spokesperson presents their blueprint in 2 minutes max. Instructor gives live feedback after each.

What students produce: - A completed six-field blueprint (Goal, Tools, Memory, Reasoning Loop, Handoff, Top Risk) - This blueprint is carried forward to Session 2 for the Failure Mode Analysis — make sure all teams save or photograph it

Instructor facilitation notes:

During the sketching phase (circulate actively):

- **Stuck on goal:** Teams sometimes write goals that are too vague ("help with marketing") or too technical ("run a sentiment analysis pipeline"). Redirect toward behavioral specificity: "What does the agent do when it wakes up? What's the first thing it looks at?"
- **Stuck on reasoning loop:** This is the hardest component. If a team is blocked, prompt them through it orally: "Okay, the agent has its goal. It starts. What's the first thing it does? It searches — and it gets back some results. Now what? What does it look at? What decision does it make? Write that sequence down." Usually unlocks within 60 seconds.
- **Over-engineered tools list:** Some teams list 10–12 tools. Push back: "If you could only give it 3 tools, which 3 matter most?" Forces prioritization and reveals the core logic of the agent.
- **Missing handoff:** Nearly every first-draft blueprint omits the handoff or makes it trivially rare ("only if something breaks"). Probe: "What if the agent drafts an email to a customer that's factually wrong? Is there a check before that sends? Who reviews it and when?" This usually produces a much more specific handoff design.
- **Top risk too abstract:** "It could make mistakes" is not useful. Push for specificity: "What kind of mistake? At which step? To whom does it cause harm?" The goal is a concrete failure scenario, not a general concern.

During the debrief (class share-out):

- Give each team exactly 2 minutes for the spokesperson. Time it. After 2 minutes, you give feedback — even if they're not done. This models the "pitch your agent" context they'll encounter in the real world.

- Feedback structure for each blueprint:
    1. **What's strong:** Identify one specific component that's well-designed. Be specific, not generic. ("The handoff trigger at the dollar threshold is exactly right — you've made a business judgment, not a technical one.")
    2. **What's underdeveloped:** Identify one component that needs sharpening. ("The reasoning loop is described as 'analyzes the data' — that's a black box. What does analyzing mean, step by step?")
    3. **The stress question:** Ask one question that sets up Session 2. ("What happens if the tool call fails and the agent gets back an error? Does it retry? Does it stop? Does it make up an answer?") Don't answer it — leave it hanging. That's intentional.
- After all teams have shared, ask the class: "What patterns do you notice across the blueprints? What did almost every team get right? What did almost every team miss?" The answer is almost always: every team defined the goal and tools well; almost no team had a robust handoff. That observation is your bridge to Session 2.

Materials needed: - Slide 11 (activity card) displayed during the full activity - Paper or access to a shared doc — no specific template required - Teams should save/photograph blueprint for use in Session 2

Timing note: If the lecture runs long (a common risk in this session given the density of the ReAct pattern explanation), protect the activity time over the debrief time. The sketching phase produces the Session 2 asset. A shorter debrief is recoverable. A rushed blueprint is not.